

Жизненные цели NPC

Аудит глубины реализации механики LifeProject в Enigma V.0.5.3.4.6

Объект:	LifeProject / LifeEngine / DriveResolver / ArchetypeLabel
Документы:	01_T3_L1_LIVING_NPC.md, 05_ONTOLOGICAL_REVOLUTIONS.md, ENTITY_CONTINUITY_CONTRACT
Код:	life_engine.py (2536 строк), life_project_resolver.py (70 строк), drive_resolver.py (60 строк), homeostasis_projecto
Логи:	67 cds_session_*.log, маркеры [LIFE_PROJECT], [BREAK_PROGRESS], [LIFE_ENGINE]
Дата:	2026-07-16

Краткий вердикт

Механика жизненных целей NPC **спроектирована детально (~100% спецификации)**, но реализована в коде лишь ~25%, а в живых сессиях реально работает ~10%. Единственный рабочий компонент — 70-строчный FSM LifeProjectResolver — встроен в тиковый конвейер, но **никогда не делал перехода состояний** в 67 записанных логах, потому что триггер identity_crisis сверху всегда False (trust/fear deltas всегда 0.0). T3 §2 (IdentityCore, DriveSystem, BodySchema, ArchetypeLabel, LivingNPC, apply_delta с инерцией, reset_recovery) **не реализован** как таковой — даже central concept «личность сопротивляется изменениям» не имеет кодовой реализации.

Компонент	Документирован	В коде	В тике	В логах
IdentityCore (frozen, rigidity, recovery_rate)	TZ §2.1	■ ОТСУТСТВУЕТ		
DriveSystem (survival/satiety/rest/social/autonomy)	TZ §2.2	■ ДРУГИЕ ДРАЙВЫ 5 ticks		
BodySchema как mutable фильтр восприятия	TZ §2.3	■ ЧАСТИЧНО 4/100 не мутирует		
ArchetypeLabel (Social/Violence enums)	TZ §2.4	■ ОТСУТСТВУЕТ		
LivingNPC + apply_delta + reset_recovery	TZ §2.5	■ ОТСУТСТВУЕТ		
LifeProject (NPCState.life_project: str)	ADR-O-315	■ ЕСТЬ 95/100 переходов FSM		
LifeProjectResolver FSM (6 состояний)	ADR-O-317	■ ЕСТЬ 0 срабатываний		
BreakProgressEngine → identity_crisis	ADR-O-316	■ ЕСТЬ никогда не True		
DriveResolver (L0+L2.5 → L3 EffectiveDrives)	ADR-O-208	■ ЕСТЬ 1078/1078 ≈const output		
Multiple Stabilizers (ENTITY_CONTINUITY §3)Контракт		■ ОТСУТСТВУЕТ		

1. Что обещают документы

1.1 T3 §01 — LivingNPC как фундамент

Файл `docs/Tasks/T3/01_T3_L1_LIVING_NPC.md` (550 строк) описывает «философскую смену парадигмы» — переход от «агент реагирует» к «агент сопротивляется». На самом первом шапке стоит пометка: «Статус: Исходная спецификация. Реализация в очереди.» T3 вводит пять сущностей, которые должны стать ядром состояния NPC:

- **IdentityCore** — frozen dataclass, генерируется один раз при рождении NPC. Поля: `base_aggression` (0.3), `base_trust` (0.5), `base_fear` (0.4), `base_curiosity` (0.3), `social_drive` (0.5), `dominance_bias` (0.2), `compliance_bias` (0.6), `rigidity` (0.9), `recovery_rate` (0.1), `resilience_multiplier` (1.0).
- **DriveSystem** — mutable, 5 драйвов: `survival`, `satiety`, `rest`, `social`, `autonomy`. Содержит `compute_stress()` и `get_dominant_drive()`.
- **BodySchema** — mutable «фильтр реальности»: `max_velocity`, `vault_capability`, `tunnel_vision_factor`, `pain_threshold`, `mana_channel`. Документ: «Травмы меняют восприятие. Стресс меняет возможности.»
- **ArchetypeLabel** — `SocialArchetype` (BROKEN/COWARD/CAUTIOUS/NEUTRAL/PROTECTIVE/LEADER) × `ViolenceArchetype` (VICTIM/SURVIVOR/AVENGER/TYRANT). Фазовый переход в конце каждого тика.
- **LivingNPC** — агрегат, содержит `core` + `drives` + `body` + `archetype` + `relationship_cache`.

Ключевой API — `apply_delta(domain, attribute, delta)`, реализующий инерцию: $new = old * rigidity + delta * (1 - rigidity)$. T3 §1.2 ссылается на CAUSAL_CONTRACT §7.1: «Личность сопротивляется изменениям. Запрещена моментальная мутация статусов.» Это центральный тезис всей механики.

1.2 Ontology Revolutions (архивный документ)

Файл `docs/ARCHIVE/2026-07-02/05_ONTOLOGICAL_REVOLUTIONS.md` (709 строк) — это философская критика текущей онтологии. Документ НЕ предлагает слой L3 life-project; ближайший концепт — `structural_states` (месяцы/годы) для `belief/faith/political_identity`. Документ предлагает 5 «революций»: `TemporalDomain` (REACTIVE/SITUATIONAL/ADAPTIVE/STRUCTURAL), `PersonalityPhaseSpace` (архетипы как точки в 3D, а не вероятности), `Narrative Time`, `CouplingBudget`, `ExpectationLoop`. Это design critique, а не implementation spec.

1.3 ENTITY_CONTINUITY_CONTRACT — авторитетная онтология

Файл `docs/ENTITY_CONTINUITY_CONTRACT.md` (Status: Accepted, Authority: Above ADR-O) определяет 5 слоёв непрерывности NPC. Слой 3 (Intent Continuity) явно говорит: «*LifeProject не является целью. Это инструмент удержания аттрактора.*» То есть жизненный проект — это не то, чего NPC хочет, а то, что удерживает его идентичность от распада. Контракт вводит **Принцип Множественных Стабилизаторов**: идентичность может опираться на несколько конкурирующих стабилизаторов (семья, статус, мастерство, безопасность, месть, принадлежность, познание). LifeProject — лишь доминирующий стабилизатор, не единственный источник смысла.

1.4 Свежие ADR онтологии (O-315 / 316 / 317)

Три ADR прямо описывают LifeProject как слой L2.7:

- **ADR-O-315 [ONTO] Internal Teleology** — вводит слой L2.7 в NPCState. `life_project` инициализируется из CoreOrientation (L0) при спавне. DecisionHub читает `state.life_direction` для буста proactive-intents. L0 строго неизменяем.

- **ADR-O-316** [ONTO] Identity Crisis & LifeProjectResolver — BreakProgressEngine возвращает только флаг `identity_crisis=True` при `stage=«deformation»`. LifeProjectResolver решает, какой новый кризис-проект выбрать (isolation/revenge). Интегрирован в `phases/decision.py`.
- **ADR-O-317** [ONTO] LifeProject State Machine — FSM из 6 состояний: ACTIVE → COLLAPSING → LOST → SEARCHING → COMMITTED → ACTIVE. LOST растит `stress` на +10 за тик до 90.0. В LOST/SEARCHING DecisionHub блокирует `life_project` бусты. `life_direction` переименован в `life_project` с `backward-compatible fallback`.

Важно: онтология использует для LifeProject метку **L2.7**, а не L3. L3 зарезервирован для EffectiveDrives — эфемерной проекции драйвов (ADR-O-208). Самый свежий онтологический ADR — **ADR-O-322** (Epistemology Machine Architecture): State → Manifestation → ObservationRelation → Signal → Fact → Hypothesis → Belief → Consumers.

2. Что реально есть в коде

2.1 LifeProjectResolver — единственный живой кусок (70 строк)

Файл `backend/app/services/npc/life_project_resolver.py` — 70 строк, реализует FSM из ADR-O-317. Это буквально вся механика жизненных целей, которая дошла до production-кода:

```
class LifeProjectResolver:
    _CRISIS_TRANSITIONS = {
        "family_builder": "isolation",
        "wealth_creator": "survival",
        "warrior": "revenge",
        "knowledge_seeker": "hermit",
        "ruler": "revenge",
        "survival": "isolation",
    }

    @staticmethod
    def resolve(state, identity_crisis, project_completed=False) -> None:
        current_state = getattr(state, "life_project_state", "ACTIVE")

        if current_state == "ACTIVE":
            if project_completed:
                state.life_project_state = "COMPLETED"
            elif identity_crisis:
                state.life_project_state = "COLLAPSING"

        elif current_state == "COMPLETED":
            state.stress = min(100.0, state.stress + 5.0)
            if state.stress >= 90.0:
                state.life_project_state = "LOST"

        elif current_state == "COLLAPSING":
            state.life_project_state = "LOST"

        elif current_state == "LOST":
            state.stress = min(100.0, state.stress + 10.0)
            if state.stress >= 90.0:
                state.life_project_state = "SEARCHING"

        elif current_state == "SEARCHING":
            current_proj = getattr(state, "life_project", "survival")
            state.life_project = LifeProjectResolver._CRISIS_TRANSITIONS.get(
                current_proj, "isolation")
            state.life_project_state = "COMMITTED"

        elif current_state == "COMMITTED":
            state.life_project_state = "ACTIVE"
```

Всё. Это вся механика LifeProject: 6 строк переходов состояний + 6 строк таблицы кризис-маппинга + инкремент stress. Никаких множественных стабилизаторов, никаких весов, никакой адаптации к контексту. Просто строка state и строка project.

2.2 Поля в NPCState

```
# backend/app/models/npc_state.py:603-607
# L2.7: LifeDirection (Жизненный проект).
# Динамическая проекция CoreOrientation (L0), модулированная убеждениями (L2.5).
# Инициализируется из L0 при спавне. Меняется при кризисе идентичности.
life_project: str = "survival"
life_project_state: str = "ACTIVE" # ADR-0-317: FSM
```

Инициализация при спавне (npc_loader.py:601) берёт CoreOrientation из L0:

```
_life_project = psyche.get("life_project",
psyche.get("life_direction",
raw_data.get("core_orientation", "survival")))
```

2.3 DriveResolver — L0+L2.5 → L3 проекция

Файл `backend/app/services/npc/drive_resolver.py` (60 строк) — чистая функция, вычисляет эфемерную проекцию драйвов на каждый тик. ADR-O-208 явно говорит: «No Projection Persistence» — без кэша, без состояния. DriveResolver клонирует `drives_base` из L0, накладывает веса CrystallizedBelief, нормализует сумму к 1.0:

```
def resolve_drives(self, archetype, beliefs=None) -> Dict[str, float]:
drives = dict(archetype.drives_base) # L0 clone
if beliefs:
for belief in beliefs: # L2.5 deformations
if belief.trait == "fear":
drives['fear'] += belief.weight * 0.5
elif belief.trait == "trust":
drives['fear'] -= belief.weight * 0.25
drives['desire'] += belief.weight * 0.125
for trait in drives:
drives[trait] = max(0.01, drives[trait]) # энтропийный пол
total = sum(drives.values())
if total > 0:
for trait in drives: drives[trait] /= total # закон сохранения Я
return EffectiveDrives.from_dict(drives)
```

Важный нюанс: драйвы в реальности — control / significance / fear / desire, НЕ те, что описаны в ТЗ (survival/satiety/rest/social/autonomy). Онтология драйвов была переработана, но ТЗ не обновлён. Это первая мажорная расхожимость документации с кодом.

2.4 LifeEngine — это НЕ про жизненные цели (2536 строк)

Название `life_engine.py` вводит в заблуждение. Этот файл на 2536 строк — макро-движок симуляции жизни: расписания NPC, маршрутизация движения, случайные события, decay стресса, need-driven движение, idle-решения. В нём **ноль** упоминаний IdentityCore, DriveSystem, BodySchema, ArchetypeLabel, LivingNPC, apply_delta, reset_recovery, life_project, life_goal, life_direction. Публичный API:

Метод	Строка	Назначение
LifeEngine.__init__(data_dir)	216	NPC cache, TemporalEngine, MovementEngine
macro_simulate(campaign_id, scene_state)	275	$state(t+\Delta)=f(state(t),\Delta)$ для долгого отсутствия
reconcile_state(campaign_id, elapsed)	425	ADR-047 аналитический decay
tick(campaign_id, scene_state)	477	Один тик: routine + needs + stress + events

tick_decisions(campaign_id, ...)	591	Phase 5 — DecisionHub per NPC (idle)
update_routine(npc, current_time, tick)	2065	Schedule → SceneChange + MovementIntent
check_random_events(npc, tick, rng)	2421	5% случайных событий
recover_stress_tick(npc)	2485	Снижение stress (сон -15, бодрствование -5)

Метод `recover_stress_tick` (строка 2485) — ближайший аналог `reset_recovery` из T3, но он снижает только поле `stress`, а не драйвы. Использует фиксированные константы `STRESS_RECOVERY_SLEEPING` и `STRESS_RECOVERY_SAFE`, без параметров `rigidity` или `recovery_rate`. **Никакой инерции личности нет.**

3. Проводка в тиковый конвейер

Цепочка вызовов для одного тика (подтверждена статически):

```
Gameloop.run_player_turn() / run_idle_tick()
└─ TickOrchestrator.execute(ctx) [tick_orchestrator.py:462]
└─ _run_core_phases(ctx) [line 526]
└─ _phase_0_simulation(ctx) [line 867]
└─ LifeEngine.tick(campaign_id, scene_state) [life_engine.py:477]
└─ _simulate_major()
└─ update_routine() [line 2065]
└─ recover_stress_tick() [line 2485] ← ~reset_recovery
└─ check_random_events() [line 2421]

└─ run_phase_0_5(ctx, deps) [tick_orchestrator.py:1257]
└─ HomeostasisProjector.compute_isolation_decay(all_npcs_raw)
[homeostasis_projector.py:36]

└─ _phase_5_decision(ctx) [tick_orchestrator.py:1095]
└─ _compute_effective_drives(npc_states, tick) [line 1060]
└─ for npc in npc_list:
└─ _profile_l0 = personality_from_legacy(npc_dict)
└─ _beliefs = self.crystallized_belief_store.get_beliefs(npc_id)
└─ _projection = self.drive_resolver.resolve_drives(_profile_l0, _beliefs)
[projection.py:1078]

└─ pipeline_runner.run(...) → evaluate_behavior_and_identity(...)
[phases/decision.py:23]
└─ for npc in npc_states:
└─ _break_deltas = BreakProgressEngine.calculate(state, willpower, ...)
[phases/decision.py:60]
└─ LifeProjectResolver.resolve(_npc_state, _break_deltas.identity_crisis,
project_completed=...) [phases/decision.py:95]
└─ NPCState.write_to_legacy(_npc_state, npc_dict) [line 100]

└─ (later) DecisionHub.compute(state, personality, event, effective_drives, ...)
└─ _base_utility(intent, event, state, ...)
└─ if event.event_type == WORLD_TICK and intent in PROACTIVE_INTENTS:
base += 0.4
if state.life_project_state in (ACTIVE, COMMITTED, COMPLETED):
_project = state.life_project
if intent in _direction_intents[_project]:
base += desire*1.5 + significance*0.5 [decision_hub.py:1440-1457]
```

Вывод: LifeProjectResolver.resolve() действительно вызывается каждый тик в phases/decision.py:95. DecisionHub читает state.life_project каждый тик для буста proactive-intents. Проводка корректна. FSM не сломан — он просто не получает триггера.

3.1 DecisionHub — как life_project влияет на решения

Код в decision_hub.py:1440-1457 определяет, какие intents бустит каждый проект:

```
_direction_intents = {
"family_builder": ["seek_ally", "help", "call_for_help"],
"wealth_creator": ["offer_job", "request_service", "trade"],
"warrior": ["ambush", "block_path", "call_for_help"],
"knowledge_seeker": ["request_service", "seek_ally"],
"ruler": ["spread_rumor", "call_for_help", "change_role"],
# Кризисные направления (L2.7)
```

```
"isolation": ["flee", "block_path"],
"revenge": ["ambush", "attack"],
"hermit": ["flee", "observe"],
}
_expected_intents = _direction_intents.get(_project, [])
if intent in _expected_intents:
    base += _desire * 1.5 + _significance * 0.5
```

Это единственное место, где `life_project` реально влияет на поведение NPC. Механика простая: если `intent` NPC совпадает с тем, что предписывает его жизненный проект, утилита получает буст $\text{desire} \times 1.5 + \text{significance} \times 0.5$. Если `life_project_state` находится в кризисной фазе (LOST, SEARCHING), буст отключается — NPC становится пассивным.

4. Что показывают логи

4.1 Частотная статистика по 67 сессионным логам

Маркер	Кол-во	Файлов	Комментарий
[LIFE_ENGINE]	588	2	Только в сессиях 0703 и 0704 — это ошибки, не успехи
[LIFE_PROJECT]	0	0	Маркер перехода FSM. Ноль срабатываний за всю историю.
[BREAK_PROGRESS]	28	1	Только ERROR'ы: name '_economic_profiles_map' is not defined
[DRIVE]	0	0	DriveResolver не логирует ничего
[HOMEOSTASIS]	0	0	HomeostasisProjector не логирует
[WILL]	0	0	WillpowerGate не логирует
[L3] / [L2.5] / [ARCHETYPE]	0	0	Таких маркеров нет в коде
life_project / life_goal / life_direction	0	0	Ни в одном логе
COLLAPSING / SEARCHING / COMMITTED	0	0	FSM состояния не появляются
identity_crisis	0	0	Триггер никогда не становится True
[PHASE_8.5] ... failed	448+	много	TickOrchestrator не имеет _get_task_scheduler
TICK_CRASH	293	14	Полные падения тика

4.2 Примеры из логов

[LIFE_ENGINE] в `cds_session_20260704_164408.log` — это ошибки целостности данных, не успешные запуски движка:

```
2026-07-04 16:44:20 app.services.npc.life_engine ERROR:
[LIFE_ENGINE][NO_POSITION] npc=merchant_goran has no location AND no position.
Activity='market_trading'. Data integrity bug – investigate npc_state.
Returning None; caller must skip movement intent.
```

[BREAK_PROGRESS] в `cds_session_20260715_220032.log` — все 28 случаев это `NameError` из-за отсутствия переменной `_economic_profiles_map` (модуль не импортирован):

```
2026-07-15 22:00:43 app.services.phases.decision ERROR:
[BREAK_PROGRESS] failed for guard_borko: name '_economic_profiles_map' is not defined
2026-07-15 22:00:44 app.services.phases.decision ERROR:
[BREAK_PROGRESS] failed for guard_borko: name '_economic_profiles_map' is not defined
... (28 раз)
```

Примечание: в текущем исходнике эта ошибка уже исправлена (строка 91 теперь комментарий), но `LifeProjectResolver.resolve()` на строке 95 — рабочий код.

[STATE_APPLIED] показывает замороженный `stress=0.0` — состояние NPC не меняется:

```
2026-07-08 21:31:06 [STATE_APPLIED] guard_borko: stress=0.0 intent=Intent.BLOCK_PATH
2026-07-08 21:31:06 [STATE_APPLIED] merchant_goran: stress=0.0
intent=Intent.REQUEST_SERVICE
2026-07-08 21:31:06 [STATE_APPLIED] maid_lusya: stress=0.0 intent=Intent.OFFER_JOB
```

4.3 Почему FSM никогда не переходит

Жизненный цикл NPC в коде построен как каскад: чтобы LifeProjectResolver сделал переход ACTIVE → COLLAPSING, нужно, чтобы identity_crisis=True. Этот флаг ставит BreakProgressEngine, когда identity_integrity падает ниже порога. А identity_integrity падает, только когда есть давление (deltas по fear/trust). По предыдущему отчёту мы знаем: **trust=+0.0 fear=+0.0 в 100% диалогов** (все 894 heard-события — NEUTRAL). Значит, давление никогда не возникает, identity_integrity никогда не падает, identity_crisis никогда не True, FSM никогда не переходит. Каскад мёртв сверху.

5. Тесты и критерии приёмки

5.1 Существующие тесты

tests/sandbox/system/test_life_direction_crisis.py — 1 тест, ПРОХОДИТ за 0.23 сек. Сценарий: maid_lusya (family_builder, low willpower) вручную переводится в глубокий кризис (identity_integrity=0.1, stress=95.0, affective_load=0.95). Проверяется, что:

- Pre-crisis: DecisionHub НЕ выбирает FLEE.
- BreakProgressEngine.calculate(state, willpower=35, recent_failures=2) → identity_crisis == True.
- Five-tick FSM walk: ACTIVE → COLLAPSING → LOST → SEARCHING (stress >= 90) → COMMITTED (life_project == «isolation») → ACTIVE.
- Post-crisis: DecisionHub выбирает FLEE или BLOCK_PATH (соответствует isolation.direction_intents).

Это канонический приёмочный тест для ADR-O-316/317. **Он проходит** — значит, FSM + consultation DecisionHub работают в изоляции. Но он требует ручной сборки NPCState с экстремальными параметрами, которых никогда не было в реальных сессиях.

5.2 Т3 §5 — критерии приёмки

Сценарий Т3	Тест существует?	Статус
test_inertia_damping (apply_delta + rigidity 0.9 → 0.14, не 0.6)	Метод apply_delta не существует	Не существует
test_cumulative_pressure (10 ударов → survival>0.5, violence=VICTIM)	Метод ArchetypeEngine не существует	Не существует
test_recovery_at_rest (reset_recovery → survival < 0.8)	Метод LivingNPCEngine не существует	Не существует

Вывод: ни один из трёх канонических приёмочных тестов Т3 §5 не существует. Это не удивительно, потому что сами сущности (IdentityCore, apply_delta, ArchetypeLabel) тоже не существуют. Т3 документирует дизайн, который так и не был реализован.

5.3 test_life_engine.py — 26 тестов, 24/26 проходят

Файл tests/test_life_engine.py тестирует LifeEngine (тот самый 2536-строчный файл макро-симуляции). 24 теста проходят, 2 падают из-за отсутствия activity_map[«sleeping»] в тестовых данных. **Ни один из этих тестов не покрывает:** apply_delta, reset_recovery, LivingNPC inertia, IdentityCore rigidity, Archetype phase transitions. Они покрывают только routine/stress-recovery API.

6. Gap analysis — что есть, чего нет, что сломано

6.1 Документировано, но НЕ реализовано

- **TZ §2.1 IdentityCore** — frozen dataclass с rigidity/recovery_rate/base_aggression/... Ни одного из этих полей нет ни в одном классе. NPCProfileL0 содержит только core_orientation: str (одна строка!) и drives_base: Dict.
- **TZ §2.2 DriveSystem** с survival/satiety/rest/social/autonomy. Реальные драйвы — control/significance/fear/desire. Совершенно другая онтология.
- **TZ §2.3 BodySchema как mutable perception filter** — существует в motion_core.py:53, но @dataclass(frozen=True) (неизменяемый!), отсутствуют concealment_volume и pain_threshold, живёт в motion-модуле (кинематический профиль), никогда не мутирует от стресса.
- **TZ §2.4 ArchetypeLabel + SocialArchetype + ViolenceArchetype** — полностью отсутствуют. 0 совпадений по grep. NPCProfileL0.archetype — это строка профессии («maid», «guard»), а не фаза личности.
- **TZ §2.5 LivingNPC + apply_delta + reset_recovery** — отсутствуют. 0 совпадений по apply_delta. Ближайший аналог — StateApplicator.apply_deltas_only, но без формулы инерции.
- **TZ §5 acceptance tests** (inertia_damping, cumulative_pressure, recovery_at_rest) — отсутствуют (см. §5.2).
- **ENTITY_CONTINUITY_CONTRACT §3 Multiple Stabilizers** — реализация поддерживает только ОДНУ строку life_project на NPC. Нет структуры для конкурирующих стабилизаторов.
- **ENTITY_CONTINUITY_CONTRACT §3 Identity Pressure Vector** (prediction_error, self_model_conflict, social_reflection_delta, value_violation) — не реализован как структурированный вектор. Ближайший аналог — NPCState.affective_load (один скаляр).

6.2 Реализовано, но НЕ документировано в ТЗ

- **LifeProjectResolver FSM** (ADR-O-317) — единственная рабочая логика жизненных целей. FSM из 6 состояний + hardcoded таблица _CRISIS_TRANSITIONS. Не упомянут в ТЗ.
- **BreakProgressEngine** — вычисляет identity_integrity_delta, pressure_resistance_delta, will_state_override, identity_crisis flag. Не в ТЗ §2.
- **DriveResolver** (L0+L2.5 → L3 EffectiveDrives) — pure function, нормализует sum=1.0. Не в ТЗ.
- **HomeostasisProjector** Phase 0.5 Field Layer — только EMA-decay social_input_ema (half-life 50 тиков). Не в ТЗ.
- **BeliefCrystallizationEngine** (L2.5, ADR-O-305) — переводит EvidenceOfPersistence в CrystallizedBelief. Не в ТЗ.
- **L1Chronicle** (ADR-O-208) — append-only TraitDriftEvent store на SQLite. Не в ТЗ.
- **CalibrationEngine** (ADR-O-211) — pass-through для стабильности L3. Не в ТЗ.
- **WillpowerGate** (ADR-031) — переводит player Intent → Pressure → WillState. Не в ТЗ (ТЗ фокусируется на NPC, WillpowerGate — на аватаре игрока).

6.3 Подключено, но НЕ срабатывает в production

- **LifeProjectResolver.resolve()** — вызывается каждый тик, но FSM никогда не переходит из ACTIVE, потому что identity_crisis никогда не True (нет trust/fear deltas).
- **BreakProgressEngine.calculate()** — выполняется без ошибок в большинстве сессий (молчит на успехе), но identity_integrity_delta предположительно ~0 (соответствует always-zero trust/fear deltas).
- **DriveResolver.resolve_drives()** — выполняется каждый тик, но даёт почти константный EffectiveDrives (драйвы дрейфуют 7 тиков и замирают — см. отчёт №1).
- **HomeostasisProjector.compute_isolation_decay()** — работает в Phase 0.5, но значимо декает social_input_ema только когда был social input. В сессиях 15-16 июля его нет (SOCIAL_EMA=0 в последних логах — см. отчёт №1).
- **BeliefCrystallizationEngine** — работает, но 0 [crystalliz] маркеров (см. отчёт №1).
- **PatternDetector** — работает, но 0 [pattern_detect] маркеров.
- **L1Chronicle** — пишет события, но query_archive падает с tuple index out of range.
- **[LIFE_PROJECT] log marker** в phases/decision.py:97 — подключен, но не срабатывает, потому что нет переходов FSM.

7. Итоговый вердикт глубины реализации

■ **ГИБРИД: ~25% кода, ~10% в живом тике** Механика жизненных целей — это не пустышка и не полностью рабочая система. Это **хорошо спроектированный, но частично реализованный** механизм, в котором рабочий FSM сидит в конце сломанной каузальной цепочки.

Слой	Дизайн	Код	В живом тике
TZ §2.1 IdentityCore	100%	0%	0%
TZ §2.2 DriveSystem	100%	0% (другие драйвы)	0% (freezes)
TZ §2.3 BodySchema	100%	~60% (frozen, 6/8 полей, не в том модуле)	0% (never mutates)
TZ §2.4 ArchetypeLabel	100%	0%	0%
TZ §2.5 LivingNPC + apply_delta + reset_recovery	100%	0%	0%
ADR-O-315 LifeProject field	100%	100% (2 строковых поля)	100% (init at spawn)
ADR-O-316 BreakProgressEngine → identity_crisis	100%	100%	~10% (runs, never True)
ADR-O-317 LifeProjectResolver FSM	100%	100% (70 строк)	0% (FSM never transitions)
ADR-O-208 DriveResolver L3	100%	100%	~30% (≈const output)
ADR-O-208 L1Chronicle	100%	100%	~20% (writes ok, reads broken)
ADR-O-305 BeliefCrystallizationEngine	100%	100%	0% (per отчёт №1)
ENTITY_CONTINUITY Multiple Stabilizers	100%	0% (одна строка)	0%
ENTITY_CONTINUITY Identity Pressure Vector	100%	0% (один скаляр)	0%

Главная цифра: ~25% совокупного дизайна (TZ + ADR-O + ENTITY_CONTINUITY_CONTRACT) реализовано как код. Из этой реализованной части **~10%** наблюдаемо срабатывает в production-сессиях — в основном потому, что триггер identity_crisis=True сверху никогда не возникает (0 ненулевых trust/fear deltas в любой записанной сессии).

У кодовой базы есть рабочий LifeProjectResolver FSM с проходящим integration-тестом, но он сидит в конце сломанной каузальной цепочки: memory writes систематически сломаны (3 независимых бага из отчёта №1), belief crystallization никогда не срабатывает, identity_integrity никогда не падает, identity_crisis никогда не True, FSM никогда не переходит, [LIFE_PROJECT] log marker имеет 0 срабатываний в 67 логах.

8. Топ-5 пробелов для закрытия

#	Пробел	Что чинит
G1	Разблокировать триггер identity_crisis (наивысший leverage). LifeProjectResolver полностью подключен, но не срабатывает, потому что BreakProgressEngine.identity_crisis всегда False. Причина: trust/fear deltas всегда 0.0 (см. отчёт №1).	Починить 3 бара memory write path: (а) передать реальный MemoryManager в npc_tick_pipeline.py:223-224 вместо None; (б) восстановить write_session_memory или убрать вызов; (в) добавить _collection_path в SqliteMemoryStore. После этого identity_integrity начнёт падать, и весь life-goal FSM станет live.
G2	Реализовать IdentityCore с rigidity/recovery_rate (TZ §2.1). Никакого параметра инерции нет. Тезис ТЗ «личность сопротивляется изменениям» не имеет кодовой реализации. Сейчас state.stress += 5.0 происходит мгновенно (см. life_project_resolver.py:47).	Добавить rigidity: float и recovery_rate: float в NPCProfileL0. Ввести apply_delta(domain, attribute, delta) на NPCState, реализующий формулу $new = old * rigidity + delta * (1 - rigidity)$. Заменить прямые мутации state.stress += в LifeProjectResolver и других местах на apply_delta.
G3	Реализовать Multiple Stabilizers (ENTITY_CONTINUITY_CONTRACT §3). Текущий NPCState.life_project: str поддерживает ОДИН стабилизатор на NPC. Контракт требует нескольких конкурирующих стабилизаторов (семья + статус + мастерство + безопасность + месть + принадлежность + познание).	Заменить life_project: str на life_projects: List[LifeProject], где LifeProject = {name, weight, state}. DecisionHub читает доминирующий (наибольший вес ACTIVE) проект. LifeProjectResolver может понижать падающий проект и повышать конкурирующий.
G4	Реализовать ArchetypeLabel фазовые переходы (TZ §2.4). SocialArchetype/ViolenceArchetype enums не существуют. DecisionHub не имеет концепции «этот NPC сейчас COWARD» или «этот NPC сейчас TYRANT» — он знает только WillState (FREE/SUBMISSIVE/BROKEN/...) и EmotionTag (NEUTRAL/fearful/...).	Добавить два enum. Добавить NPCState.archetype: ArchetypeLabel с методом update(core, drives) (по TZ §2.4 transition table). Вызывать archetype.update() в конце каждого тика. DecisionHub читает state.archetype.violence == TYRANT для attack *= 2.0 буста (TZ §3.2).
G5	Реализовать BodySchema как mutable perception filter (TZ §2.3 + §3.1). BodySchema существует в motion_core.py:53, но frozen=True, не хватает concealment_volume и pain_threshold, живёт в motion-модуле, никогда не мутирует от стресса. Тезис ТЗ «Травмы меняют восприятие. Стресс меняет возможности.» не реализован.	Либо разморозить существующий BodySchema и добавить недостающие поля, либо создать отдельный PerceptualBodySchema на NPCState. Обновлять tunnel_vision_factor при affective_load > 0.7. PerceptualKernel (или PerceptionProjector) должен читать body.tunnel_vision_factor для ослабления non-threat affordances (TZ §3.1).

Итог: жизненные цели NPC в Enigma V.0.5.3.4.6_AWC — это архитектурно зрелая концепция (ТЗ + 4 ADR + ENTITY_CONTINUITY_CONTRACT), доведённая до production-кода только в виде 70-строчного FSM. Сам FSM работает корректно (что подтверждает проходящий тест test_life_direction_crisis.py), но сидит в конце сломанной каузальной цепочки. Чтобы LifeProject начал

реально формировать поведение NPC, нужно сначала починить 3 бага memory write path (см. отчёт №1, риски R1-R3), а затем — закрыть G1 из этого отчёта (разблокировать identity_crisis триггер). Только после этого можно увидеть, как NPC меняет жизненный проект под давлением событий — и, следовательно, как из взаимодействий NPC рождается сюжет мира.